

Team Productivity Application

10 Reusable Claude Code Prompt Templates — Fill in the blanks and paste

Templates 10

Layers Backend · Frontend · Database · Full-Stack

Date July 2026

Companion to Vibe Coding Spec v1.0

LAYER TAGS →

Backend

Frontend

Database

Full-Stack

Ops / Debug

HOW TO USE

Each template has **yellow placeholders** in ALL_CAPS. Replace every placeholder before pasting into Claude Code. The annotation strip on each card tells you exactly what the template does and which files it targets.

PLACEHOLDER CONVENTION

[PLACEHOLDER_NAME] — replace the entire bracketed token including the brackets.

Example: [CONTROLLER_NAME] → task.

Prompts are designed to be self-contained after substitution.

01

New API Endpoint

Creates a fully wired Express route: controller function, route file entry, middleware, validation, and DB query

[Backend](#)

WHAT IT DOES

Scaffolds a complete endpoint from controller logic to route registration, including input validation, tenancy scoping, and error envelope.

TARGET COMPONENT

```
server/controllers/[CONTROLLER_NAME].controller.js  
server/routes/[CONTROLLER_NAME].routes.js
```

FILL IN

`[HTTP_METHOD]``[ENDPOINT_PATH]``[CONTROLLER_NAME]``[FUNCTION_NAME]``[AUTH_LEVEL]``[REQUIRED_FIELDS]``[BUSINESS_RULES]``[RESPONSE_SHAPE]`

In `team-productivity-app/server`, add a new endpoint to the existing `[CONTROLLER_NAME]` controller.

Endpoint: `[HTTP_METHOD]` `[ENDPOINT_PATH]`

Auth: `[AUTH_LEVEL]` ← one of: `none` | `JWT-any` | `JWT-PM-only`

Function: `[FUNCTION_NAME]`

Input validation (validate before any DB query; return 400 on failure):

`[REQUIRED_FIELDS]` ← e.g. `"name: required string 1-100 chars"`

Business rules (check these after validation; return appropriate 4xx if violated):

`[BUSINESS_RULES]` ← e.g. `"if sprint already active, return 409"`

Database query:

- All queries must include `AND team_id = $req.user.teamId` in `WHERE` clauses
- Use parameterised queries (never string interpolation)
- Use the existing pool from `server/config/db.js`

Success response:

`[RESPONSE_SHAPE]` ← e.g. `201: { "sprint": { id, name, status } }`

Error responses to handle:

- 400 validation failure → `{ "error": "...", "field": "fieldName" }`
- 404 resource not found → `{ "error": "Not found." }`
- 409 conflict → `{ "error": "Human-readable conflict reason." }`
- 500 unexpected DB error → `{ "error": "Something went wrong." }`

Wire the route in `server/routes/[CONTROLLER_NAME].routes.js`

applying auth and `requirePM` middleware as needed.

Do not modify any other existing routes or controller functions.

① **Tip:** For [AUTH_LEVEL], use "JWT-any" for developer-accessible endpoints and "JWT-PM-only" to have Claude automatically apply both auth + requirePM middleware in the route definition.

02

Fix Backend API Error

Diagnoses and repairs a misbehaving Express endpoint given its error output and expected behavior

Ops / Debug

WHAT IT DOES

Takes a broken endpoint, its actual vs expected behavior, and the error output — then patches only the relevant controller/route without touching unrelated code.

TARGET COMPONENT

server/controllers/[CONTROLLER_NAME].controller.js
server/middleware/[MIDDLEWARE_NAME].js (if applicable)

FILL IN

[HTTP_METHOD]

[ENDPOINT_PATH]

[ACTUAL_BEHAVIOR]

[EXPECTED_BEHAVIOR]

[ERROR_OUTPUT]

In `team-productivity-app/server`, fix the following broken endpoint.

Endpoint: [HTTP_METHOD] [ENDPOINT_PATH]

Actual behavior:

[ACTUAL_BEHAVIOR]

← e.g. "Returns 500 with 'Something went wrong' on every request"

Expected behavior:

[EXPECTED_BEHAVIOR]

← e.g. "Should return 200 with { sprint: {...} } when sprint exists"

Error output (paste the exact server log or response body):

[ERROR_OUTPUT]

Constraints:

- Fix only the code paths that cause the described failure
- Do not refactor unrelated logic in the same file
- Do not change the response shape if it is currently correct for success cases
- If the bug is in middleware (e.g. auth.js), fix it there; do not work around it in the controller
- After the fix, describe in one sentence what the root cause was

① **Tip:** Paste the exact stack trace or error JSON as [ERROR_OUTPUT]. The more specific, the less guessing Claude Code does about which code path to inspect.

WHAT IT DOES

Adds validation rules to an existing handler — checks required fields, types, lengths, and formats — and returns structured 400 errors without changing the happy-path logic.

TARGET COMPONENT

`server/controllers/[CONTROLLER_NAME].controller.js`Function: `[FUNCTION_NAME]`

FILL IN

`[CONTROLLER_NAME]``[FUNCTION_NAME]``[VALIDATION_RULES]`

In `server/controllers/[CONTROLLER_NAME].controller.js`, add input validation to the existing `[FUNCTION_NAME]` function.

Validation rules to enforce (add these at the top of the function, before any DB query):

`[VALIDATION_RULES]`

Examples:

- `name: required, string, 1-100 chars`
- `email: required, valid email format (regex or simple @ check)`
- `priority: optional, must be one of 'low' | 'medium' | 'high'`
- `endDate: required, valid ISO date, must not be in the past`

Error format for each failed rule:

Return 400 immediately on first failure:

```
{ "error": "Human-readable message.", "field": "fieldName" }
fieldName must exactly match the request body key (camelCase).
```

Constraints:

- Insert validation before line 1 of existing DB query logic
- Do not alter the success response shape
- Do not add a validation library (no Joi, Zod, express-validator) — plain JS conditionals only
- Do not extract into a helper function

Tip: "plain JS conditionals only" keeps the validation visible inline — easier to read and grade for a capstone project. If you want centralised validation later, run Template 08 (Refactor) instead.

Generates a migration script and updates any affected controller queries

WHAT IT DOES

Produces an ALTER TABLE or CREATE TABLE migration, explains the data impact, and identifies every controller file that queries the modified table so you can update them.

TARGET COMPONENT`server/db/migrate.js``server/controllers/[AFFECTED_CONTROLLER].controller.js`**FILL IN**`[TABLE_NAME]``[CHANGE_TYPE]``[COLUMN_DEFINITION]``[REASON]`

In `team-productivity-app/server/db/migrate.js`, make the following schema change.

Table: `[TABLE_NAME]`

Change type: `[CHANGE_TYPE]`

← one of: `add-column` | `add-table` | `add-index` | `add-constraint`

Definition: `[COLUMN_DEFINITION]`

← e.g. `"priority VARCHAR(10) NOT NULL DEFAULT 'medium' CHECK (priority IN ('low','medium','high'))"`

Reason: `[REASON]`

← e.g. `"Sprint needs to store end date so history page can display sprint duration"`

Deliverables:

1. Add the SQL statement to `migrate.js` so it runs when `node migrate.js` is executed
2. If adding a column to an existing table: use `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`
so the migration is safe to re-run
3. List every controller file in `server/controllers/` that `SELECT` or `INSERT` from `[TABLE_NAME]` – I will update those queries manually
4. If the new column is `NOT NULL` without a default, explain what existing rows will be set to

Do not modify any controller files – only `migrate.js`.

❶ **Tip:** Always test migrations on a dev database first. After running, verify with `psql -c "\d [TABLE_NAME]"` before touching controller files.

Scaffolds a complete page component: data fetching, loading state, error state, empty state, and role guard

WHAT IT DOES

Builds a new page with all required states (loading, error, empty, populated), wired to the correct API endpoint via the project's Axios instance, with optional PM-only gating.

TARGET COMPONENT

`client/src/pages/[PAGE_NAME]Page.jsx`

Uses: `client/src/services/api.js` ,
`AuthContext`

FILL IN

`[PAGE_NAME]``[API_ENDPOINT]``[DATA_SHAPE]``[PM_ONLY]``[RENDER_DESCRIPTION]``[EMPTY_STATE_MSG]`

In `client/src/pages/`, create a new page component: `[PAGE_NAME]Page.jsx`

Data source:

On mount, fetch: `[API_ENDPOINT]`

Expected response shape: `[DATA_SHAPE]`

← e.g. `"{ sprints: [{ id, name, startDate, endDate, tasksDone, tasksTotal }] }"`

Access control: `[PM_ONLY]`

← `"PM only - redirect developer to /dashboard"` OR `"all authenticated users"`

Render description:

`[RENDER_DESCRIPTION]`

← e.g. `"A table with columns: Sprint Name, Dates, Completed Tasks (X/Y)."`

`Each row is clickable and navigates to /retro/:sprintId/results."`

Required states:

- loading: show a centered spinner while the fetch is in flight
- error: show "Something went wrong. Please try again." with a Retry button that re-fetches
- empty: show: `"[EMPTY_STATE_MSG]"`
- data: render the UI described above

Constraints:

- Use the Axios instance from `client/src/services/api.js` (not fetch or a new axios instance)
- Use `useAuth()` from `AuthContext` for the current user's role
- Do not create new context or state management - `useState` + `useEffect` only
- Do not add `PropTypes` or `TypeScript` annotations
- Register the route in `App.jsx` under the appropriate guard (`ProtectedRoute` or `PMRoute`)

① **Tip:** Be specific in [RENDER_DESCRIPTION] — mention exact columns, click behaviors, and navigation targets. Vague descriptions produce placeholder code that still needs filling in.

Builds a focused UI component with defined props contract, no API calls, and a specified visual output

WHAT IT DOES

Creates a stateless or lightly-stateful presentational component with an explicit props contract. Keeps all data fetching in parent pages — this component only renders and emits callbacks.

TARGET COMPONENT

```
client/src/components/[COMPONENT_NAME].jsx
```

FILL IN`[COMPONENT_NAME]``[PROPS_CONTRACT]``[VISUAL_DESCRIPTION]``[CALLBACKS]`

In `client/src/components/`, create `[COMPONENT_NAME].jsx`

Props contract:

`[PROPS_CONTRACT]`

← e.g.

- ← `items: array of { id, title, status } — required`
- ← `onSelect: (id) => void — called when user clicks a row`
- ← `selected: array of ids — controls checkbox state`
- ← `disabled: boolean — if true, all interactions are blocked`

Visual description:

`[VISUAL_DESCRIPTION]`

← describe layout, colors, truncation rules, badges, etc.

Callbacks emitted:

`[CALLBACKS]`

← e.g. `"onSelect(id) — called on row click; onConfirm([ids]) — called on confirm button click"`

Constraints:

- ← This component must NEVER call `api.js` directly — all data comes in as props
- ← No `useEffect`, no data fetching, no direct API calls
- ← If the component needs to show a loading state, accept a `loading` prop (boolean)
- ← Use inline styles or plain CSS classes — no CSS module or `styled-component`

imports

- ← Export as default

❶ **Tip:** The "never calls `api.js`" constraint is intentional — it keeps components testable in isolation and prevents data fetching scattered across the component tree. `DashboardPage` owns data; `KanbanBoard` only renders it.

WHAT IT DOES

Identifies the broken React state update, missing dependency, or incorrect conditional render, and applies a minimal surgical fix without restructuring the component.

TARGET COMPONENT

```
client/src/[pages|components|views]/[COMPONENT_FILE].jsx
```

FILL IN

`[COMPONENT_FILE]``[SYMPTOM]``[STEPS_TO_REPRODUCE]``[EXPECTED_BEHAVIOR]`

In `client/src/[pages|components|views]/[COMPONENT_FILE].jsx`, fix the following bug.

Symptom:

`[SYMPTOM]`

← e.g. "Task card does not update its column after drag-and-drop until page reload"

Steps to reproduce:

`[STEPS_TO_REPRODUCE]`

← e.g.

1. Log in as Developer
2. Open /dashboard with active sprint
3. Drag a card from To Do to In Progress
4. Card snaps back to To Do visually even though the API returns 200

Expected behavior:

`[EXPECTED_BEHAVIOR]`

← e.g. "Card should remain in the In Progress column immediately after drop"

Console errors (paste any browser console errors or React warnings):

`[CONSOLE_OUTPUT]`

Constraints:

- Fix only the broken behavior — do not refactor surrounding code
 - Do not change the component's props contract or state structure unless necessary
 - If the fix requires a change to a parent component (e.g. `DashboardPage`), make it there
- and note clearly what you changed and why
- After the fix, explain in one sentence what the root cause was

① **Tip:** Most drag-and-drop visual bugs come from optimistic state updates not being applied before the API returns. If you see a snap-back, check whether the state update is inside or outside the API callback.

WHAT IT DOES

Replaces hardcoded data in a page with a real API call using the project's Axios instance, adds loading/error/empty states, and maps the response shape to the component's expected props format.

TARGET COMPONENT

`client/src/pages/[PAGE_NAME]Page.jsx`Uses: `client/src/services/api.js`

FILL IN

`[PAGE_NAME]``[API_ENDPOINT]``[API_RESPONSE_SHAPE]``[STATE_VARIABLE]``[COMPONENT_PROP]`

In `client/src/pages/[PAGE_NAME]Page.jsx`, wire the existing page to the live API.

Current state: the page renders with hardcoded / placeholder data or an empty array.

API endpoint to call: `[API_ENDPOINT]`

→ e.g. `"GET /api/sprints/history"`

API response shape:

`[API_RESPONSE_SHAPE]`

→ e.g. `"{ sprints: [{ id, name, startDate, endDate, tasksDone, tasksTotal, retroSubmitted, retroTotal }] }"`

Mapping to component state:

Store response data in state variable: `[STATE_VARIABLE]`

Pass to child component as prop: `[COMPONENT_PROP]`

→ e.g. `"Store in 'sprints' state; pass as sprints prop to SprintHistoryTable"`

Required states to add:

- `loading=true` while fetch is in flight → render spinner
- `error` → render "Something went wrong." with a Retry button (re-calls the endpoint)
- `empty` (data returned but array is empty) → render existing empty-state message
- `data` → render existing UI with live data

Constraints:

- Use the Axios instance from `client/src/services/api.js`
- Call the API in `useEffect` with an empty dependency array (fetch on mount only)
- Do not change the JSX render structure – only add the data fetching and states
- Do not add a loading library or `suspense`

① **Tip:** "Do not change the JSX render structure" prevents Claude from rewriting a working UI while adding a fetch call. If the render structure also needs updating, run a separate prompt for that change.

Adds a feature visible and functional only to PM users — both in the frontend UI and the backend API

WHAT IT DOES

Implements a new PM-only capability end-to-end: conditionally renders UI based on role, guards the route with `PMRoute`, and applies `requirePM` middleware on the corresponding backend endpoint.

TARGET COMPONENT

`client/src/pages/[PAGE_NAME]Page.jsx` (UI gate)
`server/controllers/[CONTROLLER].controller.js` (API gate)
`client/src/App.jsx` (route gate)

FILL IN

[FEATURE_NAME]

[UI_LOCATION]

[UI_ELEMENT]

[API_ENDPOINT]

[CONTROLLER_FUNCTION]

Add a new PM-only feature: [FEATURE_NAME]

Frontend changes:

Location: [UI_LOCATION]

← e.g. "In `RetroResultsPage.jsx`, below the response list"

UI element to add: [UI_ELEMENT]

← e.g. "A 'Close Retro' button that opens a confirm dialog before calling the API"

Gate the UI element with:

```
{ user.role === 'pm' && ( ...UI element... ) }
```

Import `useAuth` from `AuthContext` to get `user.role`.

Backend changes:

Endpoint: [API_ENDPOINT]

Controller function: [CONTROLLER_FUNCTION]

Apply `requirePM` middleware in the route definition (after `auth` middleware).

Route guard:

If the feature requires a new page/route, add it to `client/src/App.jsx` under the `PMRoute` wrapper (not `ProtectedRoute`).

Verification steps to run after implementation:

1. Log in as PM → confirm the UI element is visible and functional
2. Log in as Developer → confirm the UI element is completely absent from the DOM
3. As Developer, call [API_ENDPOINT] directly from DevTools fetch → confirm 403
4. Confirm no console errors in either session

❗ **Tip:** Always enforce the guard at BOTH layers (UI conditional AND backend `requirePM`). A UI-only gate is not security — a developer user can still call the endpoint directly if the API is unprotected.

Generate a Manual Test Script for an Endpoint

Produces a ready-to-run sequence of `fetch()` calls that tests an endpoint's happy path, edge cases, and auth boundaries

Ops / Debug

WHAT IT DOES

Generates a Node.js or browser `fetch()` test script that covers: success case, validation errors, auth failure (no token), role failure (wrong role), and any domain-specific edge cases like duplicate submission or locked sprint.

TARGET COMPONENT

Tests: `[HTTP_METHOD]` `[ENDPOINT_PATH]`

Run in: Node.js terminal or browser DevTools console

FILL IN

`[HTTP_METHOD]`

`[ENDPOINT_PATH]`

`[SUCCESS_PAYLOAD]`

`[PM_TOKEN]`

`[DEV_TOKEN]`

`[EDGE_CASES]`

Generate a manual test script for: `[HTTP_METHOD]` `[ENDPOINT_PATH]`

Script format: Node.js using built-in `fetch` (Node 20+). Each test is a self-contained `async` function. Print `PASS` or `FAIL` with the actual response for each test.

Tokens to use (paste real JWTs from a running dev server):

PM token: `[PM_TOKEN]`

Developer token: `[DEV_TOKEN]`

Base URL: `http://localhost:5000`

Test cases to generate:

1. Happy path – valid payload, PM token

Payload: `[SUCCESS_PAYLOAD]`

Expect: `2xx` status + response body described in spec

2. Missing required field – remove one required field from the payload

Expect: `400 + { error: "...", field: "fieldName" }`

3. No auth token – omit Authorization header entirely

Expect: `401 + { error: "Authentication required." }`

4. Wrong role – use Developer token on PM-only endpoint (skip if endpoint is public)

Expect: `403 + { error: "Access denied." }`

5. Edge cases specific to this endpoint:

`[EDGE_CASES]`

– e.g.

– "Duplicate submit: call endpoint twice with same user, expect 409 on second"

```
call"

- "Completed sprint: attempt status update on locked sprint, expect 403"
- "Cross-team access: use a token from team A to request a resource from team B"

Output format for each test:
console.log("TEST N - [description]: PASS") or FAIL + actual response body
```

- ① **Tip:** The cross-team access test in [EDGE_CASES] is the most important security test in this project. Always include it for any endpoint that returns or modifies team-scoped data — tenancy bugs are silent until they aren't.